

Multimodal Models

Petra Galuscakova

petra.galuscakova@uis.no



Department of Electrical Engineering and Computer Science
University of Stavanger

March 3, 2026



Vision Transformers

UNet

Diffusion Model



- ▶ Classical pipeline:
 - CNNs dominate vision (ResNet, EfficientNet)
 - Transformers dominate language (BERT, GPT)
- ▶ But multimodal models require a unified representation space for text and images
- ▶ Key problem: CNNs rely on local inductive bias while transformers rely on global self-attention
- ▶ **Can we treat an image like a sequence of tokens?**



- ▶ Transformers perform poorly on vision at small scale, but outperform CNNs when trained on massive datasets.
- ▶ Google developed JFT-300M dataset: a large-scale, internal computer vision dataset consisting of 300 million images labeled with 18,291 categories
- ▶ Vision is not fundamentally different from language — scale may dominate inductive bias.



- ▶ The ViT idea: An image = a sequence of patches = tokens.
- ▶ Instead of convolutions over pixels
- ▶ We do:
 - Linear patch embedding
 - Positional encoding
 - Standard transformer encoder
- ▶ This aligns vision with NLP architectures, multimodal fusion and diffusion cross-attention conditioning



Vision Transformers

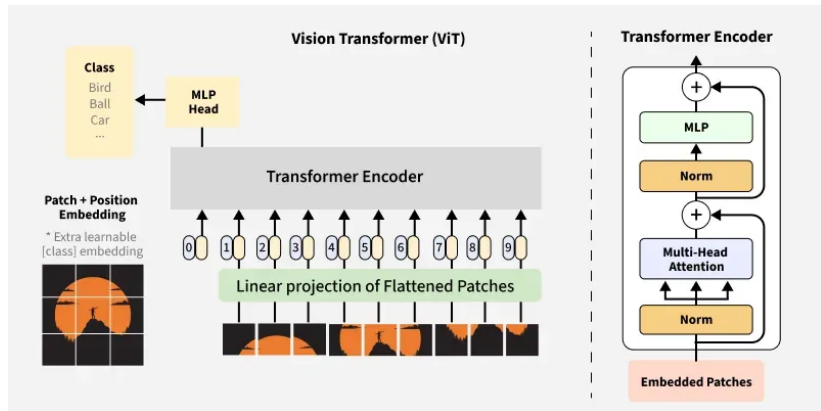
UNet

Diffusion Model



- ▶ Instead of relying on convolutions, ViTs use self-attention to capture relationships across all image patches, enabling a global understanding of the image.
- ▶ Instead of processing words, ViT treats an image as a sequence of fixed-size patches and applies self-attention across them.
- ▶ This allows the model to capture long range dependencies between different parts of an image without relying on convolution operations.
- ▶ This approach has achieved state-of-the-art results in various computer vision tasks (image classification, object detection and segmentation).

Vision Transformer (ViT) Architecture Overview



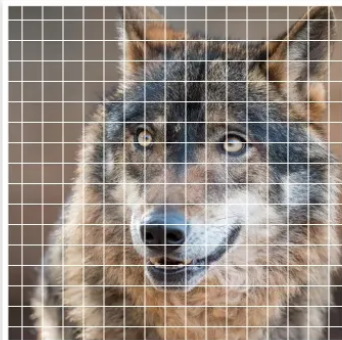


- ▶ This stage converts a 2D image into a sequence of patch embeddings, analogous to tokens in NLP.
- ▶ **Patch Splitting:** The image is divided into fixed size and non-overlapping patches each treated as a token and converted into a 1D sequence.
- ▶ **Patch Flattening:** Each patch of size $P \times P \times C$ is flattened into a single vector. This flattening removes spatial dimensions temporarily and allows the model to treat patches uniformly.
- ▶ **Patch Embedding (Linear Projection):** Each flattened patch is mapped to a learnable D dimensional embedding enabling the model to learn high level features similar to word embeddings.
- ▶ Patch embeddings can also be **extracted** using a convolution layer, making each convolution act as a patch extractor.

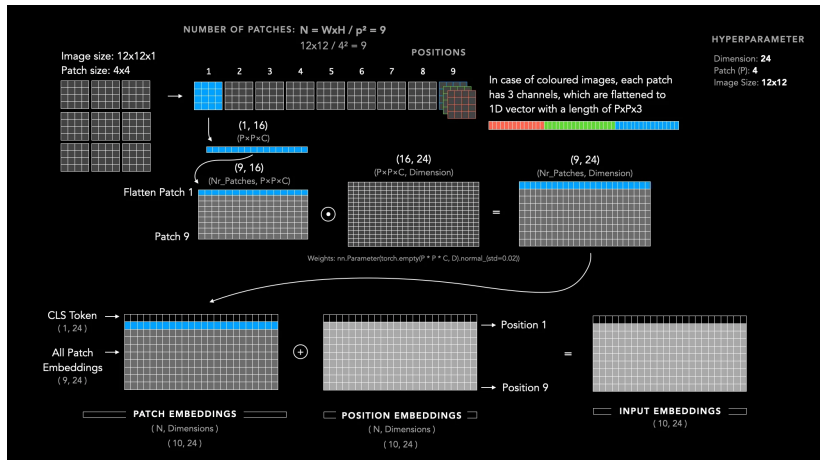
Original Image



Split Image (Patches)



Patches to Embeddings



1

¹<https://medium.com/advanced-deep-learning/understanding-vision-transformers-vit-70ca8d817ff3>

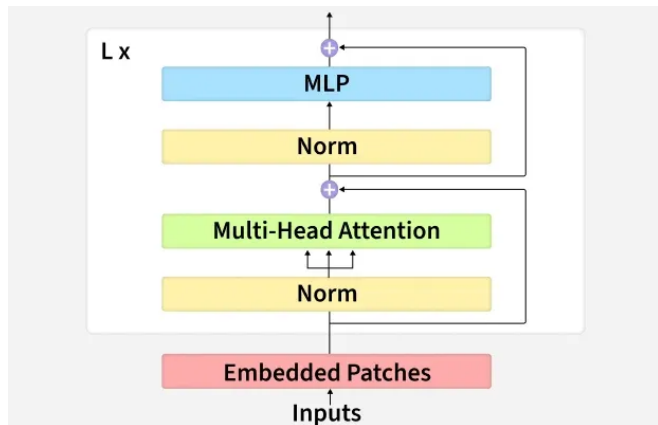


- ▶ **Need for Positional Encoding:** Since Transformers treat tokens as unordered, positional encodings are added to retain spatial structure and patch location information.
- ▶ **Learnable Positional Embeddings:** ViT uses learnable positional vectors to capture local and global spatial relationships adapting better than fixed encodings across image resolutions.



- ▶ A learnable CLS token is prepended to the patch sequence to aggregate information from all patches, serving as the image-level representation for classification.
- ▶ **How the CLS Token Learns Image-Level Representation:** The CLS token attends to all patches to learn global image features and its final output alone is used for prediction without CNN style pooling.
- ▶ Once the image is split into patches and the CLS token is added, these patches with CLS token pass through the Transformer's layers. The key part of this process is self-attention: each token attends to every other token, allowing the model to learn the relationships between patches.

Transformer Encoder (Pre-LayerNorm Architecture)



The Pre-LayerNorm (Pre-LN) Transformer architecture is a design modification of the original Transformer where the Normalization Layer is applied before the Multi-head attention and feed-forward sub-layers, rather than just after the residual addition.



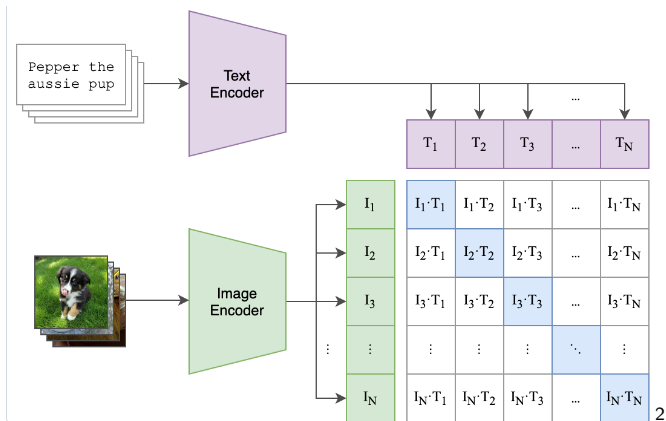
- ▶ This stabilizes gradient flow and prevents the exploding/vanishing gradient problem in deep Transformers.
- ▶ $\text{LayerNorm}(x) = x - \mu\sigma \odot \gamma + \beta$ where
 - μ, σ are mean and std across features
 - γ, β are learnable.
- ▶ Pre-LN ensures that the activations processed by the attention and feed-forward networks have a consistent scale (zero mean, unit variance) regardless of the depth.
- ▶ The gradients flowing backward through the network are also generally better behaved, as the normalization step effectively "resets" the scale at the input of each residual block.



- ▶ ViTs need more data than CNNs due to low inductive bias and training involves pretraining on large datasets followed by finetuning.
- ▶ **Inductive Bias Differences:** CNNs use strong inductive biases like locality and translation invariance, while ViTs treat images as patch sequences, requiring more data but offering greater flexibility.
- ▶ **Pretraining:** Pretraining lets ViTs learn general visual features via supervised or self-supervised methods, reducing compute needs for downstream tasks.
- ▶ **Finetuning:** Finetuning adapts pretrained ViTs to specific datasets using fewer labels, often with layer-wise learning rate decay to improve performance.



- ▶ Multimodal models require aligning images and text in a shared embedding space.
- ▶ Contrastive Language–Image Pretraining (CLIP) learns this alignment.
- ▶ It trains two encoders:
 - Image encoder (often ViT)
 - Text encoder (Transformer)
- ▶ The model maximizes similarity between matching image–text pairs and minimizes similarity between mismatched pairs.
- ▶ This creates a joint representation space where text and images become geometrically aligned.



²<https://towardsdatascience.com/clip-model-and-the-importance-of-multimodal-embeddings-1c8f6b13bf72/>



- ▶ CLIP is trained using a contrastive loss.
- ▶ For a batch of N image–text pairs, the objective encourages correct pairs to have high cosine similarity.
- ▶ The loss can be written as:
$$L = - \sum_{i=1}^N \log \frac{\exp(\text{sim}(I_i, T_i)) / \tau}{\sum_{j=1}^N \exp(\text{sim}(I_i, T_j)) / \tau}$$
- ▶ This forces semantic alignment between modalities.



- ▶ Vision Transformers learn powerful global image representations.
- ▶ However, classification is not generation.
- ▶ Generative models must:
 - Model the full data distribution
 - Produce new samples
- ▶ Question: How do we move from understanding images to generating them?



Vision Transformers

UNet

Diffusion Model

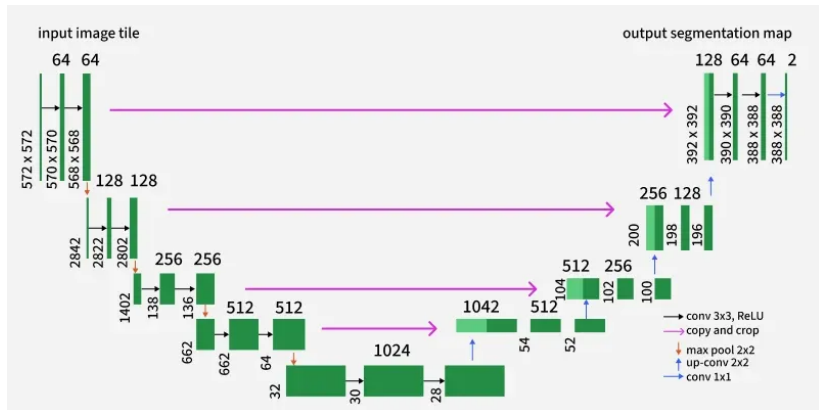


- ▶ Generative models require predicting fine-grained pixel-level details.
- ▶ The architecture must:
 - Capture global structure
 - Preserve spatial information
 - Use high-frequency details
- ▶ U-Net provides:
 - Multi-scale representation (i.e. representation at different level of detail)
 - Skip connections for detail recovery (the key innovation).
 - ▶ This means the decoder receives both semantic features from deep layers and fine spatial features from early layers
 - Strong spatial inductive bias (nearby pixels are related and patterns repeat across space)



- ▶ U-Net is a kind of neural network proposed for image segmentation which means dividing an image into different parts to identify specific objects for example separating a tumor from healthy tissue in a medical scan.
- ▶ The name 'U-Net' comes from the shape of its architecture which looks like the letter 'U' when drawn.
- ▶ It is widely used in medical imaging because it performs well even with a small amount of labeled data.
- ▶ The U-Net architecture has evolved from its original purpose of image segmentation and is currently used in generative AI.

UNet Architecture



The image shows U-Net turning a 572×572 image into a smaller 388×388 segmented map. It shrinks the image to capture features then upsamples to restore size using skip connections to keep details. The output labels each pixel as object or background.



- ▶ The architecture is symmetric and has three key parts:
- ▶ 1. Contracting Path (Encoder):
 - Uses small filters (3×3 pixels) to scan the image and find features.
 - Apply RELU activation function to add non-linearity.
 - Uses max pooling (2×2 filters) to shrink the image size while keeping important information. This helps the network focus on bigger features.
- ▶ 2. Bottleneck:
 - The middle of the “U” where the most compressed and abstract information is stored. It links the encoder and decoder.



► 3. Expansive Path (Decoder):

- Uses upsampling i.e increasing image size to get back the original image size.
- Combines information from the encoder using “skip connections.” These connections help the decoder get spatial details that might have been lost when shrinking the image.
- Uses convolution layers again to clean up and refine the output.



- ▶ **Input Image:** The process starts by feeding a medical or other input image typically grayscale into the network.
- ▶ **Feature Extraction (Encoder):** The encoder extracts increasingly abstract features by applying convolutions and downsampling. At each level the spatial size decreases while the number of feature channels increases and allow the model to capture higher-level patterns.
- ▶ **Bottleneck Processing:** This is the middle part of the network where the image is reduced the most. It holds a small but very meaningful version of the image that captures the main features.
- ▶ **Reconstruction and Localization (Decoder):** The decoder begins to reconstruct the original image size through upsampling. At each level it combines decoder features with corresponding encoder features using skip connections to retain fine-grained spatial details.



- ▶ **Skip Connections for Precision:** Skip connections help preserve spatial accuracy by bringing forward detailed features from earlier layers. These are especially useful when the model needs to distinguish boundaries in segmentation tasks.
- ▶ **Final Prediction:** A 1×1 convolution at the end converts the refined feature maps into the final segmentation map where each pixel is classified into a specific class like foreground or background. This output has the same spatial resolution as the input image.



- ▶ U-Net was originally designed for segmentation.
- ▶ But its architecture is well suited for a different task:
 - Predicting noise
 - Reconstructing corrupted signals



- ▶ In diffusion models, U-Net predicts the noise added at each diffusion step.
- ▶ The input consists of:
 - Noisy image x_t
 - Timestep embedding t
 - (Optional) Text embedding for conditioning
- ▶ The output is a noise estimate $\epsilon_{\theta}(x_t, t)$
- ▶ This transforms U-Net from a segmentation model into a denoising backbone.



<https://www.geeksforgeeks.org/machine-learning/u-net-architecture-explained/>



Vision Transformers

UNet

Diffusion Model



- ▶ Diffusion models are a type of generative AI that create new data like images, audio or even video by starting with random noise and gradually turning it into something meaningful.
- ▶ They work by simulating a diffusion process where data is slowly corrupted by noise during training and then learning to reverse this process step by step.
- ▶ By doing so the model learns how to generate high quality samples from scratch.

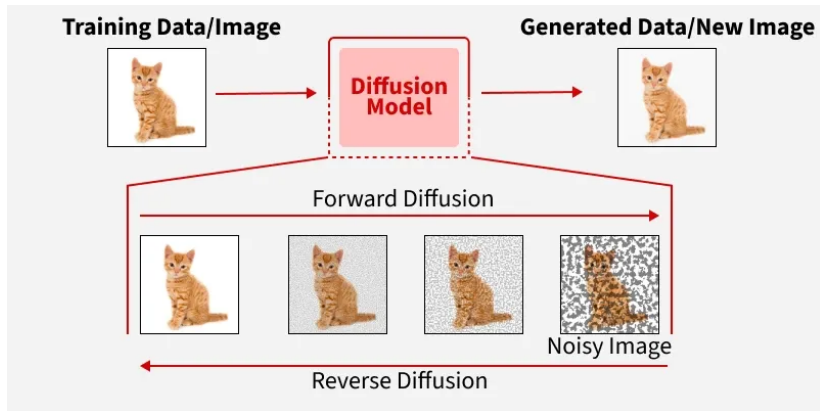


- ▶ Earlier generative models relied on adversarial training.
- ▶ Generative Adversarial Networks (GANs) introduced instability:
 - Mode Collapse
 - Difficult optimization
 - Sensitive hyperparameters
- ▶ Diffusion models reformulate generation as reversing noise.
- ▶ Essentially these models operate as a reverse diffusion phenomenon where noise is introduced to the data in a forward manner and removed in a reverse manner to generate new data samples.
- ▶ By learning to reverse this process diffusion models start from noise and gradually denoise it to produce data that closely resembles the training examples.



<https://pinetools.com/add-noise-image>

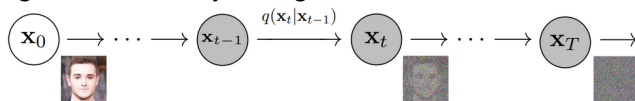
Understanding Diffusion Models





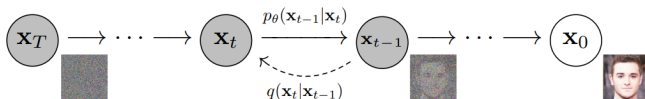
- ▶ **Forward Diffusion Process:** This process involves adding noise to the data in a series of small steps. Each step slightly increases the noise, making the data progressively more random until it resembles pure noise.
- ▶ **Reverse Diffusion Process:** The model learns to reverse the noise-adding steps. Starting from pure noise, the model iteratively removes the noise, generating data that matches the training distribution.

- In this process noise is incrementally added to the data over a series of steps. This is akin to a Markov chain where each step slightly degrades the data by adding Gaussian noise.



- Let x_t represent the noisy data at time step t . The process is defined as: $x_t = \sqrt{1 - \beta_t}x_{t-1} + \beta_t\epsilon$ where β_t is the noise schedule that controls the amount of noise added at each step and ϵ is Gaussian noise.

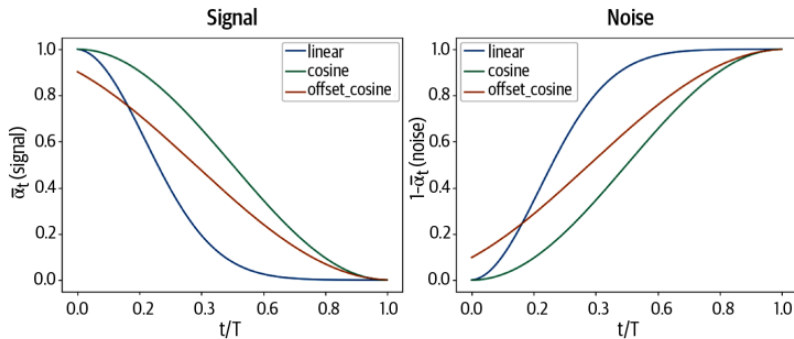
- ▶ The reverse process aims to reconstruct the original data by denoising the noisy data in a series of steps reversing the forward diffusion.



- ▶ The reverse process aims to reconstruct the original data x_θ from the noisy data x_T at the final time step T .



- ▶ During training the model learns to predict the noise added at each step of the forward process.
- ▶ The training objective for diffusion models involves minimizing the difference between the true noise ϵ added in the forward process and the noise predicted by the neural network ϵ_θ .
- ▶ The loss function is typically the mean squared error (MSE) between these two quantities: $L(\theta) = \mathbb{E}[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$

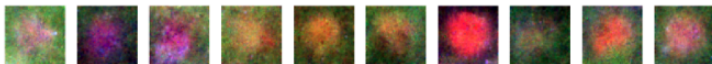


Diffusion Steps

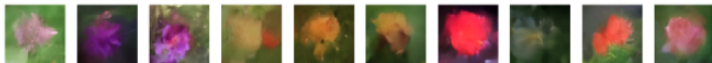


Diffusion
steps

1



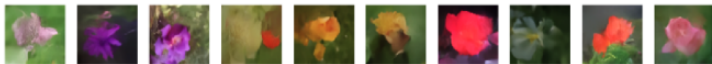
2



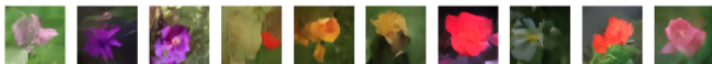
3



4



5



20



100





- ▶ Diffusion can be performed in pixel space or latent space.
- ▶ Latent Diffusion compresses images using a Variational Autoencoder (VAE): $x \rightarrow \text{Encoder} \rightarrow z$
- ▶ Diffusion operates in latent space z instead of pixel space.
- ▶ Advantages:
 - Lower memory usage
 - Faster training



- ▶ Pixel Diffusion:
 - Operates directly on image pixels
 - Higher fidelity
 - Higher compute cost
 - Used in Imagen
- ▶ Latent Diffusion:
 - Operates in compressed space
 - Much more efficient
 - Enables open-source deployment
 - Used in Stable Diffusion



- ▶ So far, diffusion generates images unconditionally, multimodal systems should generate images conditioned on text.
- ▶ This requires:
 - Multimodal alignment
 - Text embeddings
 - Conditioning mechanisms
- ▶ The key idea: Text modifies the denoising trajectory.



- ▶ <https://www.geeksforgeeks.org/artificial-intelligence/what-are-diffusion-models/>



- ▶ **Image Generation:** Diffusion models are widely used to generate realistic images from random noise. This application is especially popular in fields like art, gaming, advertising and graphic design where high quality visuals are essential.
- ▶ **Image Editing:** They enable advanced editing by filling in missing or damaged parts of an image. This is useful in photo restoration, object removal or editing specific regions without affecting the whole image.
- ▶ **Super Resolution:** Diffusion models can improve the quality of low resolution images by enhancing details. This application benefits medical imaging, satellite photos and surveillance footage.



- ▶ **Slow Sampling:** Generating samples can be slow because of the many steps needed for the reverse diffusion process.
- ▶ **Complexity:** The architecture and training process can be complex making them challenging to implement and understand.
- ▶ **Memory Usage:** High memory consumption during training due to the need to store multiple intermediate steps.
- ▶ **Fine Tuning:** Requires careful tuning of noise schedules and other hyperparameters to achieve optimal performance.